

basics.py

```
1
2 print("Hello, world!")
3 #print() displays output on the screen.
4
5 name = "Alice"
6 age = 25
7 height = 5.6
8 #No need to declare types
9 #Python figures it out automatically
10
11 x = 10          # int
12 y = 3.14        # float
13 name = "John"   # string
14 is_student = True # boolean
15
16 #Basic Operators
17 a = 10
18 b = 3
19
20 print(a + b) # addition
21 print(a - b) # subtraction
22 print(a * b) # multiplication
23 print(a / b) # division
24 print(a % b) # remainder
25 print(a ** b) # exponentiation
26
27 #Conditional Statements
28 age = 18
29 if age >= 18:
30     print("Adult")
31 else:
32     print("Minor")
33
34 #Loops
35 for i in range(5):
36     print(i)
37
38 count = 0
39 while count < 5:
40     print(count)
41     count += 1
42
43 #Lists
44 fruits = ["apple", "banana", "cherry"]
45 print(fruits[0]) # Accessing elements
46 fruits.append("orange") # Adding an element
47 print(fruits)
48
```

```
49 #Functions
50 def greet(name):
51     return f"Hello, {name}!"
52
53 print(greet("Alice")) # Calling the function
54
55 #user input
56 name = input("Enter your name: ")
57 print(f"Hello, {name}!")
58
59 -----
60
61 #Dictionaries
62 #Dictionaries store key-value pairs. They are unordered, mutable, and indexed by keys.
63
64 person = {
65     "name": "Alice",
66     "age": 25,
67     "city": "New York"
68 }
69 print(person["name"]) # Accessing values
70 person["age"] = 26 # Modifying values
71 print(person.get("age")) # 26
72 print(person)
73 person.pop("grade") # removes "grade"
74 for key, value in person.items(): # looping through dictionary items
75     print(key, value)
76
77 -----
78 #Sets
79 #Sets are unordered collections of unique elements. They are mutable and do not allow
80 duplicate values.
81 numbers = {1, 2, 3, 3, 4}
82 print(numbers) # {1, 2, 3, 4}
83 numbers.add(5) # Adding an element
84 print(numbers) # {1, 2, 3, 4, 5}
85 numbers.remove(2) # Removing an element
86 print(numbers) # {1, 3, 4, 5}
87 numbers.remove(2) # error if not found
88 numbers.discard(10) # no error if not found
89 # Looping through set
90 for n in numbers:
91     print(n)
92 # Set operations
93 A = {1, 2, 3}
94 B = {3, 4, 5}
95
96 print(A | B) # union → {1,2,3,4,5}
97 print(A & B) # intersection → {3}
98 print(A - B) # difference → {1,2}
```

```

98 print(B - A) # difference → {4,5}
99
100 #| Feature      | Dictionary          | Set                |
101 #| -----      | -----            | -----            |
102 #| Structure     | key → value         | just values        |
103 #| Duplicates    | keys unique         | no duplicates      |
104 #| Order         | ordered (Python 3.7+) | unordered          |
105 #| Example       | {"a": 1}            | {1, 2, 3}          |
106
107 #Use dictionary when:
108 #You need to store related data
109 #👉 e.g., student info, user profiles
110 #Use set when:
111 #You need unique values only
112 #Fast membership checking
113 #👉 e.g., removing duplicates
114 -----
115 #tuples
116 #Tuples are ordered, immutable collections of elements. They can contain duplicate values and
  are indexed by position.
117 numbers = (1, 2, 3)
118 names = ("Alice", "Bob", "Charlie")
119 print(numbers[0]) # 1
120 print(numbers[-1]) # 3
121 numbers = (1, 2, 3)
122 numbers[0] = 10 # ❌ Error!
123 #👉 You cannot modify a tuple after creation.
124 #💡 Single-element tuple (common mistake)
125 single = (5) # This is just an integer, not a tuple
126 print(type(single)) # <class 'int'>
127 single_tuple = (5,) # This is a single-element tuple
128 print(type(single_tuple)) # <class 'tuple'>
129
130 #🔄 Looping through tuple
131 for num in numbers:
132     print(num)
133
134 #📦 Tuple unpacking (very useful)
135 person = ("Alice", 25)
136
137 name, age = person
138
139 print(name) # Alice
140 print(age) # 25
141
142 #Common tuple methods
143 t = (1, 2, 3, 2)
144
145 print(len(t)) # 4
146 print(t.count(2)) # 2

```

```

147 print(t.index(3)) # 2
148
149 # Nested tuples
150 data = ((1, 2), (3, 4))
151
152 print(data[0])      # (1, 2)
153 print(data[0][1])   # 2
154 #| Feature      | Tuple      | List      |
155 #| -----      | -----    | -----    |
156 #| Mutability    | ❌ Immutable | ✅ Mutable  |
157 #| Speed        | Faster     | Slightly slower |
158 #| Syntax       | ( )       | [ ]       |
159
160 #Use tuples when:
161 #You want an ordered collection of items
162 #The data should not change (immutability)
163 #👉 e.g., coordinates, RGB value
164 # Use lists when:
165 #You need a mutable collection of items
166 #👉 e.g., to-do list, shopping car
167

```